Brand Mention and Review Agent

Project plan v2: daily collection of posts and reviews, AI replies, human approval, and posting back to the platform

Platforms	Instagram, Facebook, LinkedIn, Reddit, Google reviews
Daily job	Runs automatically every day at a fixed time; results stored by date
Dashboard	Every item shows platform, content, link, AI reply, Approve / Deny
On approval	Agent posts the reply under the post or review on that platform
Version	v2 - 14 October 2026 (replaces the 8-page v1)

1. Project overview

An agent runs once every day at a set time and collects every post, comment or review that tags or mentions the company on Instagram, Facebook, LinkedIn, Reddit and Google. Each item is saved under that day's date and shown on a dashboard with its platform, text, a link to the original and an AI-written reply. A human presses **Approve** or **Deny**. On approval the agent posts the reply under the original item on its platform.

What the system must do

- **Collect daily** from all five platforms at a scheduled time, without gaps if a run fails.
- **Store by date** so any day's items can be fetched with a date filter.
- **Generate an AI reply for every item** (business enquiries get the team link; job posts get the careers portal and HR email; reviews get a thank-you or an apology).
- **Show everything on a dashboard:** platform, post or review, link, AI reply, Approve / Deny.
- **Post the approved reply** to the platform and record the result.
- **Keep an audit trail** of who approved what and when.

What changed from v1

v1	v2
Real-time webhooks for tags	Scheduled daily collection (webhooks can be added later as a bonus)
Facebook, Instagram, LinkedIn	Adds Reddit and Google reviews
Dashboard inbox	Dashboard organised by scrape date, with a date picker
Approve / edit / reject	Approve / Deny (plus optional edit), then post to the platform

Platform summary

Platform	What is collected	How it is collected	How the reply is posted
Instagram	Media and comments that tag or @mention the account	Meta Graph API (tags and mentions)	API reply to the comment or mention
Facebook	Posts and comments that tag or mention the Page	Meta Graph API	API comment; some cases may need manual posting
LinkedIn	Posts and comments that mention the company Page	Community Management API	API comment, if approved for that action
Reddit	Posts and comments that mention the brand (search by brand terms)	Reddit API search	API comment from the company account (see risks)
Google	Reviews of the company's own business locations	Business Profile API	Official owner reply to the review

2. How the data is actually collected (read this first)

The brief says 'scrape'. For the big platforms, true scraping (a bot reading web pages) is the weakest foundation for a business tool, so this plan uses three collection methods in order of preference:

Method	Used for	Why
1. Official API	Instagram, Facebook, LinkedIn, Google, Reddit	Allowed by the platform, stable, and the only route that also lets the agent post replies.
2. Licensed data provider	Wider internet coverage (other review sites, public mentions not tagged to your accounts)	Paid service that supplies the data legally. Optional, Phase 5.
3. Direct page scraping	Not planned	Breaks often, risks account bans and legal claims. Only for sites whose terms allow it, after legal review.

Why not scrape Meta and LinkedIn directly? Meta states that scraping violates its Terms and has taken legal action against companies doing it, including a settlement with a permanent injunction in 2022. LinkedIn's user agreement also prohibits automated data collection. A scraper could get the company's pages restricted, which would also break the posting feature. Official APIs avoid this.

Two limits to plan around

- **Official APIs return what is tagged to your own accounts.** They do not search the whole internet. A post that names the company in plain text without tagging it may be missed on Meta and LinkedIn. Reddit is the exception: it supports keyword search.
- **Google reviews are only available for business locations you manage.** The API is for owners and managers, not for reading reviews of other businesses.
- **Posting is not equally possible everywhere.** Each platform gets a 'post mode': *automatic* (API) or *assisted* (the dashboard opens the post and copies the approved reply so a human pastes it). This keeps the Approve button useful even where API posting is refused.

3. User flow

#	Step	Who	Detail
1	Daily trigger	Scheduler	Fires at the set time (for example 02:00 IST). Creates a ScrapeRun for today's date.
2	Collect	Collectors	One collector per platform fetches new items since the last successful run.
3	Save by date	Agent	Items saved with scrape_date = today; duplicates skipped.
4	Classify and draft	LLM	Category plus an AI reply for every item.
5	Dashboard	Human	Opens a date (default today), reads items with platform, link and AI reply.
6	Decide	Human	Approve or Deny; may edit the reply text before approving.
7	Post	Agent	On Approve, posts the reply under the item on its platform and stores the result.
8	Follow up	Human	Business enquiries get a call-back; job posts need no action.

Item status lifecycle

```
NEW -> REPLY_DRAFTED -> PENDING_REVIEW --(Approve)--> APPROVED --> POSTED
                                |                        |-> POST_FAILED (retry)
                                |                        |-> MANUAL_POST_NEEDED -> POSTED
                                |--(Deny)--> DENIED
```

Reply rules

Item type	AI reply content	Extra dashboard action
Business enquiry	Friendly reply with the link to connect with the team.	Mark for call-back; show author details for verification.
Job / career	We would love to connect; check the careers portal; if no vacancy fits, contact HR at the HR email.	None.
Positive review / praise	Short thank-you.	Bulk approve option.
Negative review / complaint	Apology, invitation to contact support (written by the stronger model).	Flag as priority; edit encouraged.
Spam / irrelevant	No reply drafted.	Auto-archive; weekly spot check.

4. Daily schedule and date-based storage

Every item carries two time fields: when it was originally posted, and the date it was collected (scrape_date). The dashboard filters on scrape_date, so selecting 12 October shows exactly what the 12 October run collected.

- **Scheduler:** Celery Beat (or a cron job) triggers the run at a fixed time. Use one time zone (Asia/Kolkata) for scrape_date so dates never shift.
- **No gaps:** each collector fetches 'everything since the last successful run', not 'the last 24 hours'. If a run fails or the server is down, the next run catches up automatically.
- **No duplicates:** a unique key on (platform, platform_item_id) means re-fetching an item never creates a second row.
- **Per-platform isolation:** if Reddit fails, the other four still finish; the run is marked 'partial' and the failed platform is retried.
- **Run log:** a ScrapeRun record stores start and end time, status and item counts per platform, shown on the dashboard as 'Last run: 12 Oct 02:00, 41 new items'.
- **Run now button:** lets an admin trigger a collection manually.
- **Fetch by date:** GET /api/items?date=2026-10-12&platform=reddit&status=pending.

Collection flow

```
Scheduler (daily, fixed time)
   -> create ScrapeRun(run_date)
   -> for each platform: collector.fetch(since = last_success)
        -> normalise -> save Item(scrape_date = run_date)
   -> queue classification + AI reply for every new Item
   -> close ScrapeRun (success / partial / failed), alert on failure
```

5. Dashboard specification

Each row (or card) is one scraped post, comment or review.

Element	What it shows
Platform	Icon and name: Instagram, Facebook, LinkedIn, Reddit or Google
Type	Post, comment, mention or review (with star rating for Google)
Content	The post or review text, author name and profile link

Element	What it shows
Link	Button that opens the original post or review on its platform
Scraped on	Scrape date and time; original post date
AI response	The generated reply, editable in place; category badge and confidence
Approve	Marks the reply approved and triggers posting
Deny	Rejects the reply; asks for an optional reason; nothing is posted
Status	Pending, approved, posted, denied, post failed, manual post needed

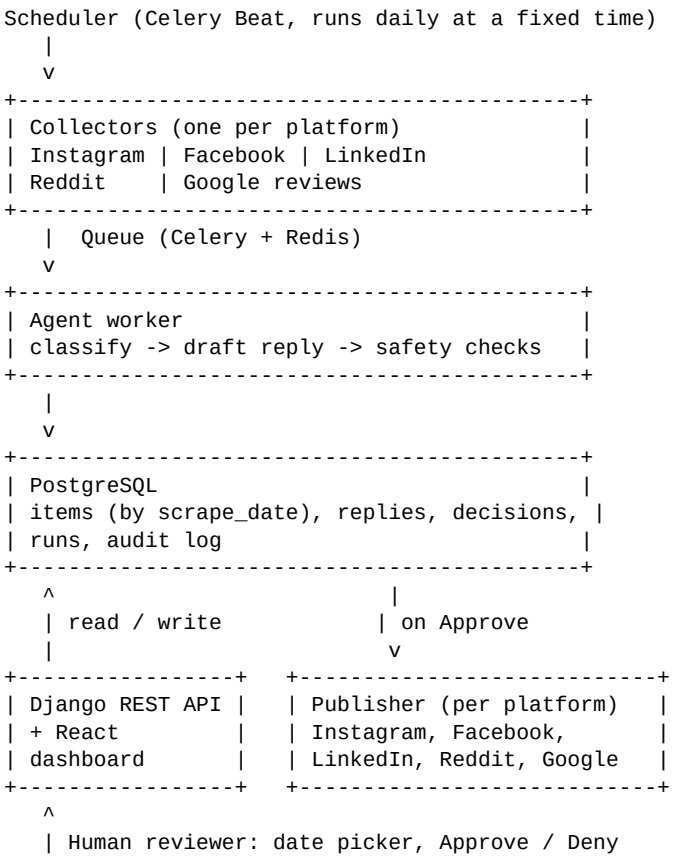
Screens and filters

- **Daily view (home):** date picker defaulting to today, previous / next day arrows, and counts (new, pending, approved, denied).
- **Filters:** platform, category, status, rating (reviews), search text.
- **Item detail:** full text, link, author, classifier reasoning, reply history, audit trail.
- **Leads:** business enquiries marked for call-back, with contact status and notes.
- **Runs:** history of daily runs with status and per-platform counts.
- **Settings:** connected accounts, brand search terms for Reddit, team link, careers URL, HR email, post mode per platform (automatic or assisted), schedule time.

Approve behaviour

- Approve is final for that reply; the posted text is stored exactly as sent.
- If the platform refuses the post (permissions, rate limit, deleted post), the item becomes POST_FAILED with the error shown and a Retry button.
- In assisted mode, Approve changes the item to MANUAL_POST_NEEDED and offers Open post and Copy reply; the human confirms with Mark as posted.
- A posting rate limit (for example a few replies per hour per platform) prevents bulk approvals from looking like bot activity.

6. Architecture design



Components

Component	Technology	Responsibility
Scheduler	Celery Beat or cron	Starts the daily run; retries failed platforms.
Collectors	Python; Meta Graph API, LinkedIn API, Reddit API, Google Business Profile API	Fetch new items since last success and normalise them.
Agent worker	Python, LLM API with JSON output	Classify, draft reply, run safety checks.
Database	PostgreSQL	Items (indexed by scrape_date), replies, decisions, runs, tokens, audit.
API	Django REST Framework	Date-filtered item lists, approve / deny / edit, run-now.
Dashboard	React + Vite	Daily view, filters, approve / deny, leads, settings.
Publisher	Per-platform poster classes	Post approved replies; record URL or error.
Secrets	Environment or secrets manager	App secrets and refreshable tokens, encrypted.

Core data model

Table	Key fields
ScrapeRun	run_date, started_at, finished_at, status (success / partial / failed), per-platform counts and errors
Item	platform, platform_item_id (unique with platform), kind (post / comment / review / mention), author name / handle / url, text, rating, item_url, source_created_at, scraped_at, **scrape_date**, run, category, confidence, status
Reply	item, text, version, generated_by (AI / human), model, prompt_version
Decision	reply, reviewer, action (approve / deny / edit), reason, decided_at, final_text
PostResult	reply, mode (api / manual), posted_at, platform_comment_url, error
SocialAccount	platform, account or location id, encrypted token, token expiry, active
Config	team link, careers URL, HR email, reddit search terms, schedule time, post mode per platform
AuditLog	actor, action, object, timestamp

Key design decisions

- **Links never come from the LLM.** The team link, careers URL and HR email are inserted by code from Config, so the model cannot invent them.
- **Human approval for every reply** before anything is posted.
- **Posts and reviews are untrusted text.** They are passed to the LLM as data, the LLM has no tools, and its output is validated (allowed categories, length, no extra links).
- **Platform differences hidden behind two interfaces:** Collector.fetch(since) and Publisher.post(reply). Adding a new platform means adding one class of each.
- **Index on scrape_date and platform** so date filtering stays fast as the table grows.

7. LLM strategy: demo first, production later

The work is short-text classification and short reply writing, so speed and cost matter more than the largest model. Confirm current model names and prices on the provider's page before committing.

Demo

- **Claude Haiku 5.5** through the Anthropic API for both classification and reply drafting: cheap, fast and reliable at strict JSON output.
- For zero spend, a free tier from another provider can be used for experiments, accepting rate limits and changing terms.
- Keep the model behind one llm_client wrapper so switching is a configuration change.

Production

Task	Suggested model	Why
Classification and extraction	Claude Haiku 5.5	High volume, short text, low cost.
Routine replies (praise, enquiries, jobs)	Claude Haiku 5.5 plus templates	Templates carry the fixed content; the model personalises the wording.
Negative reviews, complaints, low-confidence items	Claude Sonnet 5.5	Tone matters most where a poor public reply does damage.
Fallback	Second provider behind the same wrapper	Avoids a single point of failure.

Cost and quality

- Every item needs a reply, so cost scales with daily volume. At a few hundred items a month the LLM bill is typically a few dollars; measure real usage during the demo.
- Use a fixed system prompt, short inputs and prompt caching where supported.
- Build a labelled test set of 100 to 200 items across all five platforms; track classification accuracy and how often humans edit the AI reply.
- Store the model name and prompt version with each reply for later comparison.

Output format (example)

```
{
  "category": "business_enquiry",
  "confidence": 0.93,
  "reason": "Asks for a quote for an AI chatbot",
  "reply": "Thanks for reaching out! You can connect with our team here: {TEAM_LINK}"
}
```

8. Things to do before starting

Approvals take the longest and are outside your control. Start them first and build against mock data while you wait.

A. Business and legal

- Confirm the company owns and will let you connect its Instagram, Facebook Page, LinkedIn Page, Reddit account and Google Business Profile.
- Get management approval for automated collection, AI-written public replies and storing public post data.
- Ask legal or management to confirm the approach (official APIs; no direct scraping of Meta or LinkedIn).
- Decide the reply links: team contact URL, careers portal URL, HR email.
- Prepare a public privacy policy page and a company website (needed for app reviews).
- Decide data retention: how long items and author details are kept, and who can see them.

B. Meta (Instagram and Facebook)

- Create a Meta developer account and app; set Instagram to a Professional account linked to the Facebook Page.
- Complete Business Verification.

- Request only the permissions needed for mentions, comments and replies; prepare the App Review screencast showing a human approving each reply.
- Test with tester accounts in Development mode; switch to Live after approval; plan long-lived token refresh.
- Note: story mentions are not available through the Mentions API.

C. LinkedIn

- Create a developer app tied to the company Page; a super admin of the Page must verify it.
- Apply for Community Management API access (Development tier), then Standard tier with a screen recording.
- Prepare legal name, registered address, website and privacy policy.
- Development tier call limits are small, so test sparingly.

D. Google reviews

- The Business Profile must be verified and active for 60 days or more, and have a website listed.
- Create a Google Cloud project and submit the Business Profile API access request using an email that is an owner or manager of the profile.
- Approval is visible as a quota above zero in the Cloud Console; then enable the API.
- Only your own locations are available; reviews of other businesses cannot be read.

E. Reddit

- Register a Reddit app and read Reddit's current Data API terms before building: commercial use may require a paid or approved arrangement. One 2026 source (sponsored content, unverified) reports a high commercial price, so ask Reddit directly.
- Check subreddit rules: many communities restrict bots or brand self-promotion.
- Plan to start in assisted mode (a human pastes the reply) and move to automatic posting only if the account stays in good standing.
- Agree the brand search terms (company name, product names, common misspellings).

F. Technical setup

- Repository, branching workflow and issue tracker.
- Django, PostgreSQL, Redis, Celery and Celery Beat; React and Vite dashboard.
- A public HTTPS URL for OAuth callbacks (a tunnel in development, a domain in production).
- Secrets management; tokens encrypted; nothing committed to git.
- LLM API key with a spending alert; logging and error alerts from day one.

9. Phased roadmap

Phase	Deliverable	Depends on
0 - Prerequisites	Account access; Meta, LinkedIn, Google and Reddit applications submitted; links and policy page ready.	Management sign-off
1 - Core on mock data	Data model with scrape_date, daily scheduler, classifier, reply drafter, dashboard with date picker and Approve / Deny, assisted posting. Test set and accuracy report.	Nothing external
2 - Google + Reddit	Collectors for Google reviews and Reddit; Google owner reply posting.	Google access; Reddit terms
3 - Instagram + Facebook	Meta collectors and reply posting.	Meta approval
4 - LinkedIn	LinkedIn collector and comment posting.	LinkedIn approval
5 - Hardening and extras	Retries, rate limits, monitoring, roles, backups; optional licensed data provider for wider internet coverage; optional real-time webhooks.	Earlier phases

Estimated running cost (low volume)

Item	Estimate
Meta, LinkedIn, Google Business Profile APIs	No usage fee found in the documentation reviewed; approvals required. Confirm before launch.
Reddit API	Unclear for commercial use; ask Reddit. Budget as an open item.
LLM	A few dollars per month at a few hundred items; measure in the demo.
Licensed data provider (optional)	Paid; varies by provider and volume.
Hosting, database, Redis	Small server; similar to your other projects.

Prices and API terms change often. Confirm each figure on the provider's official page before budgeting.

Risks and mitigations

Risk	Mitigation
App review rejected or delayed	Minimal permissions, clear screencast, submit early, build on mock data meanwhile.
Official APIs miss untagged mentions	State the limit; add a licensed data provider later if coverage matters.
Posting refused on some platforms	Assisted mode: open post, copy approved reply, human pastes.
Reddit bans the account or subreddits object	Assisted mode first, read subreddit rules, low reply volume.
Bad or off-brand public reply	Human approval, templates, validated links, stronger model for complaints.
Daily run fails silently	Catch-up from last success, run log, alert on failed or partial runs.
Token expiry breaks collection or posting	Scheduled refresh and alerts before expiry.
Prompt injection in post text	Treat text as data, no tools for the LLM, strict output validation.
Account flagged for automation	Rate-limit posting, vary wording, human-approved replies only.

Open decisions

- Who owns each platform account and who will use the dashboard?

- What time should the daily run happen, and should there be a second run (for example evening)?
- Is a licensed data provider acceptable for wider internet coverage?
- Which categories, if any, may auto-post in the future?
- Where will the app be hosted?

Sources checked

Meta on scraping and enforcement: about.fb.com/news/2020/10/taking-legal-action-against-data-scraping. Meta settlement coverage: techcrunch.com/2022/10/03/meta-settles-lawsuit-for-significant-sum-against-businesses-scraping-facebook-and-instagram-data. Google Business Profile API prerequisites: developers.google.com/my-business/content/prereqs. LinkedIn Community Management app review: learn.microsoft.com/en-us/linkedin/marketing/community-management-app-review. Instagram Mentions API limits: singhamandeep.com/?p=1566. Reddit pricing: techloy.com (sponsored, unverified; check Reddit's own terms).